

WorkWise

A three-sided job board on MERN: candidates build one profile and apply with it, recruiters post roles and move applicants through a pipeline, admins keep the whole thing honest. Auth is done. Everything the product is actually *for* is not.

STACK	REPOS	TARGET	EST. TO MVP
Express 5 · Mongoose 9 · React 19 · Vite	node-auth · node- auth-frontend	Real users, public launch	6-8 focused weeks

CONTENTS

- 01 Where it stands
- 02 The product
- 03 MVP scope
- 04 Data model
- 05 API surface

- 06 Build order
- 07 Going live
- 08 Decisions
- 09 What will bite

Where the project stands

An honest read of both repos as they are today. The pattern is clear: authentication is genuinely finished, and the `User` schema describes a rich candidate profile that no endpoint can write to.

AREA	STATE	DETAIL
Register / login	BUILT	bcrypt cost 12, httpOnly cookies, 15 min access + 7 day refresh, rotation on refresh.
Password reset	PARTIAL	Token generation, hashing and 15 min expiry are correct — but the token is <code>console.log</code> ed. No mail provider, so no user can actually reset.
Email verification	MISSING	<code>isEmailVerified</code> exists on the schema. Nothing ever sets it to <code>true</code> .
Roles & guards	BUILT	<code>authorizeRoles("ADMIN")</code> works. <code>RECRUITER</code> is in the enum but no route uses it.
Candidate profile	MISSING	The single biggest gap. Skills, experience, education, resume, salary expectations are all modelled and all unreachable — no read route, no write route. <code>getProfile</code> is written but never mounted on the router.
Jobs	MISSING	No model, no routes. The frontend board renders an empty state from <code>src/data/jobs.js</code> .
Applications	MISSING	No model. The bookmark button on <code>JobCard</code> holds state in React and forgets on refresh.
Admin console	PARTIAL	Directory, stats, search, filters and profile drawer all work — read-only, because <code>GET /auth/users</code> is the only admin route that exists.
Validation	PARTIAL	Hand-rolled <code>if (!x)</code> checks per controller. No schema validation, so no consistent 400 shape.
Rate limiting	MISSING	Login, register and forgot-password are all unthrottled. This is the first thing that gets abused on a public URL.
File uploads	MISSING	<code>resume.url</code> has no way to be populated.
Tests / CI	MISSING	No test runner in either repo.

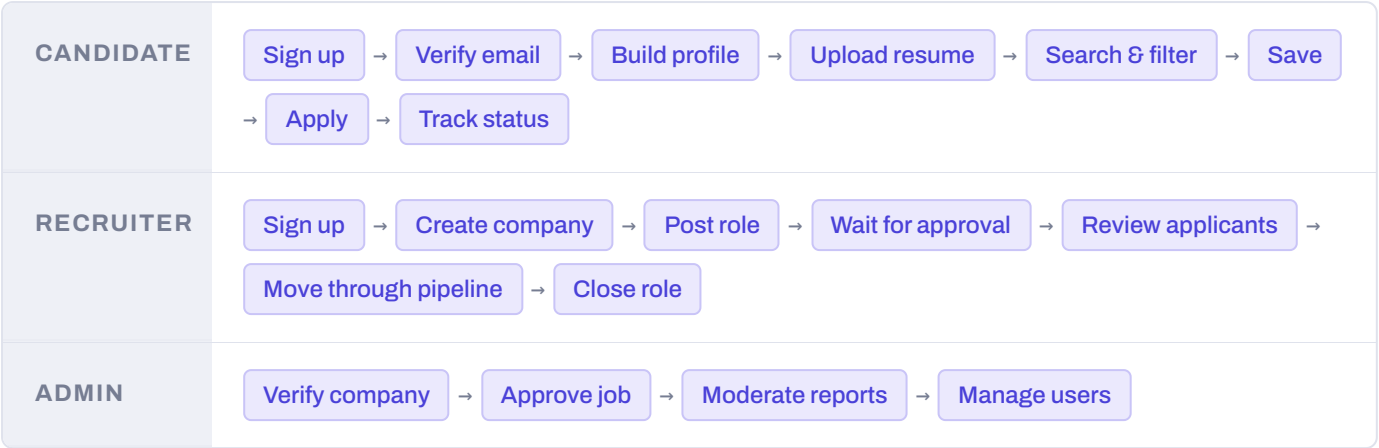
TWO SMALL THINGS WORTH FIXING NOW

`refreshTokens` stores raw token strings. If the database ever leaks, every session is forgeable — hash them the same way you already hash reset tokens.

`crypto` is listed in `package.json`. That npm package is a deprecated stub; Node's core module wins the resolution anyway, so it does nothing but confuse the next reader. Remove it.

The product

Three actors, one loop. The product only works when all three sides turn — a board with no recruiters has nothing to show, and a board with no candidates gives recruiters nothing back.



The admin lane exists because you are launching to real users. An open posting form on a public job board attracts scams within days — fake roles harvesting phone numbers and ID documents. Approval before a job goes live is not bureaucracy here, it is the product's defence, and it is cheap while volume is low.

MVP scope

MVP means the loop closes end to end for one real recruiter and one real candidate. Anything that does not sit on that path waits.

In

- **Accounts** — register as candidate or recruiter, verified email, working password reset, session revocation.
- **Candidate profile** — every field the schema already defines, plus a resume file, editable and readable.
- **Companies** — one company per recruiter to start, admin-verified.
- **Jobs** — create, edit, publish, pause, close. A status lifecycle with admin approval before **ACTIVE**.
- **Search** — keyword, location, work mode, employment type, experience band; sorted and paginated server-side.
- **Applications** — apply once per job, withdraw, track status. Recruiter moves applicants through a pipeline; candidate is emailed on change.
- **Saved jobs** — persistent, not React state.
- **Admin** — the write actions the console is already laid out for, plus verification and moderation queues, all audit-logged.

Out — deliberately

- In-app messaging or chat between recruiter and candidate. Email is enough at launch.
- Resume parsing, match scoring, recommendations. Every one of these is a project of its own and none of them is why someone signs up.
- Payments, featured listings, subscriptions. Monetise after the loop turns.
- Interview scheduling, video, calendars.
- Company pages with reviews and ratings — that is Glassdoor, not an MVP.
- Multiple recruiters per company, org invites, seat management. One owner per company until someone asks.
- Mobile app, i18n, dark mode for the product itself.

Data model

Five collections. `User` exists and needs additions; the other four are new. Every reference is stored as an `ObjectId` with a `ref`, and every list query below has an index behind it.

User EXTEND

Keep what you have. These are the additions that email verification, session management and lockout need.

FIELD	TYPE	WHY
emailVerificationToken	String (hashed)	Same pattern as the reset token you already wrote.
emailVerificationExpires	Date	24 hours is a sane window.
refreshTokens[]	[{ tokenHash, userAgent, ip, createdAt, expiresAt }]	Replaces <code>[String]</code> . Lets you show "signed-in devices" and revoke one, not all.
failedLoginAttempts	Number	Lockout counter.
lockUntil	Date	Temporary lock after repeated failures.
lastLoginAt	Date	Useful to you, and to the admin directory.
company	ObjectId → Company	Set for <code>RECRUITER</code> only. Null for everyone else.
deletedAt	Date	Soft delete. Never hard-delete a user who has applications attached.

INDEXES

```
{ email: 1 } unique • already there
{ role: 1, createdAt: -1 } • admin directory listing
{ emailVerificationToken: 1 } sparse
{ resetPasswordToken: 1 } sparse
```

Company NEW

A job belongs to a company, not to a string. This is what makes "all roles at X" possible later, and it is where the trust signal lives — `verified` is the gate that stops a scam listing.

FIELD	TYPE	NOTES
name	String, required	
slug	String, unique	URL identity. Generate from name, dedupe with a suffix.

FIELD	TYPE	NOTES
website	String	Part of how you verify them.
logo	String (url)	Falls back to initials, which the UI already does.
industry	String	
size	Enum	1-10 · 11-50 · 51-200 · 201-500 · 500+
about	String, max 2000	
locations[]	[String]	
owner	ObjectId → User	The recruiter who created it.
verified	Boolean, default false	Jobs cannot go ACTIVE while this is false.
verifiedAt / verifiedBy	Date / ObjectId → User	Audit trail.

INDEXES

```
{ slug: 1 } unique
{ verified: 1, createdAt: -1 } · admin verification queue
```

Job NEW

Field names match what the frontend already renders, so **JobCard**, **FilterRail** and the filter helpers in **data/jobs.js** keep working when you swap the local array for a fetch.

FIELD	TYPE	NOTES
title	String, required	
slug	String, unique	Include a short id — job titles repeat constantly.
company	ObjectId → Company	
postedBy	ObjectId → User	
description	String, required	Store as markdown, render sanitised.
responsibilities[] / requirements[]	[String]	Bulleted lists beat one wall of text.
category	String	Engineering, Design, Data...
skills[]	[String], lowercased	Normalise on write or filtering will betray you.
location	{ city, state, country }	Structured beats a free-text string for filtering.
workMode	Enum	REMOTE · HYBRID · ONSITE
employmentType	Enum	FULL_TIME · PART_TIME · CONTRACT · INTERNSHIP

FIELD	TYPE	NOTES
experience	{ min, max } Number	Years. Drives the experience-band filter.
salary	{ min, max, currency, period, isDisclosed }	Store numbers, format in the UI. <code>isDisclosed: false</code> renders "Not disclosed".
openings	Number, default 1	
screeningQuestions[]	[{ question, type, required }]	Answers live on the application. Keeps you off a separate collection.
status	Enum	<code>DRAFT</code> • <code>PENDING_REVIEW</code> • <code>ACTIVE</code> • <code>PAUSED</code> • <code>CLOSED</code> • <code>EXPIRED</code>
applicantCount	Number, default 0	Denormalised counter, updated with <code>\$inc</code> . Counting applications on every list query will not scale.
viewCount	Number, default 0	
publishedAt / expiresAt	Date	Expire at 30 days unless renewed. Stale listings are the fastest way to lose candidate trust.

INDEXES

```

{ title: "text", description: "text", skills: "text" } • keyword search
{ status: 1, publishedAt: -1 } • the default board query
{ status: 1, workMode: 1, employmentType: 1 } • facet filtering
{ company: 1, status: 1 } • recruiter's own listings
{ slug: 1 } unique • { expiresAt: 1 } • expiry sweep

```

Application NEW

The one model people get wrong. It must snapshot the candidate at the moment they applied — if someone edits their headline or swaps their resume next month, the recruiter must still see what they actually received.

FIELD	TYPE	NOTES
job	ObjectId → Job	
candidate	ObjectId → User	
company	ObjectId → Company	Denormalised so a recruiter can list across all their jobs in one query.
status	Enum	<code>APPLIED</code> • <code>IN_REVIEW</code> • <code>SHORTLISTED</code> • <code>INTERVIEW</code> • <code>OFFERED</code> • <code>REJECTED</code> • <code>WITHDRAWN</code>
resumeSnapshot	{ url, fileName }	Copy at apply time, not a live reference.
profileSnapshot	{ name, email, phone, headline, totalExperience, skills[] }	Same reason.

FIELD	TYPE	NOTES
coverLetter	String, max 3000	
answers[]	[{ question, answer }]	Against the job's screening questions.
statusHistory[]	[{ status, changedBy, note, at }]	Powers the candidate's tracker and settles disputes.
viewedByRecruiterAt	Date	"Application viewed" is the single most reassuring thing you can show a candidate.

INDEXES

{ job: 1, candidate: 1 } unique · one application per person per job, enforced by the database

{ candidate: 1, createdAt: -1 } · candidate's tracker

{ job: 1, status: 1 } · recruiter's pipeline

{ company: 1, createdAt: -1 } · all applicants for a company

SavedJob NEW

Its own collection rather than an array on the user, so you can sort by when it was saved and page through it without loading the whole user document.

FIELD	TYPE	NOTES
candidate	ObjectId → User	
job	ObjectId → Job	
createdAt	Date	From <code>timestamps: true</code> .

INDEXES

{ candidate: 1, job: 1 } unique · saving twice is a no-op

{ candidate: 1, createdAt: -1 } · the saved list

Also worth adding once the above works

- `AuditLog` — { actor, action, targetType, targetId, meta, ip, at } . Every admin write goes through it. You will want this the first time someone asks why an account was disabled.
- `Notification` — in-app notification feed, if email alone starts to feel thin.

API surface

Everything under `/api` . **BUILT** marks what already runs today.

Auth — `/api/auth`

METHOD	PATH	ACCESS	STATE
POST	/register	Public	BUILT
POST	/login	Public	BUILT
POST	/logout	Public	BUILT
POST	/logout-all	Auth	BUILT
POST	/refresh	Cookie	BUILT
GET	/me	Auth	BUILT
POST	/forgot-password	Public	NEEDS MAIL
POST	/reset-password/:token	Public	BUILT
POST	/change-password	Auth	BUILT
POST	/verify-email/:token	Public	NEW
POST	/resend-verification	Auth	NEW
GET	/sessions	Auth	NEW

Profile — `/api/me`

METHOD	PATH	ACCESS	PURPOSE
GET	/profile	Auth	Full own profile.
PATCH	/profile	Auth	Whitelisted fields only.
POST	/experience	Candidate	Add an entry.
PATCH / DELETE	/experience/:id	Candidate	Edit, remove.
POST	/education	Candidate	Add an entry.
PATCH / DELETE	/education/:id	Candidate	Edit, remove.
POST	/resume	Candidate	Upload; returns url + fileName.
DELETE	/resume	Candidate	

METHOD	PATH	ACCESS	PURPOSE
DELETE	/account	Auth	Soft delete. Required if you take real users' data.

Jobs — /api/jobs

METHOD	PATH	ACCESS	PURPOSE
GET	/	Public	Search. Query: <code>q</code> , <code>location</code> , <code>workMode[]</code> , <code>type[]</code> , <code>expMin</code> , <code>expMax</code> , <code>sort</code> , <code>page</code> , <code>limit</code> . Returns <code>ACTIVE</code> only.
GET	/:slug	Public	Job detail; increments <code>viewCount</code> .
POST	/	Recruiter	Create as <code>DRAFT</code> .
PATCH	/:id	Owner	Edit own job.
POST	/:id/publish	Owner	→ <code>PENDING_REVIEW</code> , or straight to <code>ACTIVE</code> if the company is verified.
POST	/:id/close	Owner	
GET	/:id/applications	Owner	The pipeline.
POST	/:id/apply	Candidate	Verified email required.
POST / DELETE	/:id/save	Candidate	Bookmark toggle.

Applications, companies, admin

METHOD	PATH	ACCESS	PURPOSE
GET	/api/me/applications	Candidate	Tracker, filterable by status.
GET	/api/me/saved-jobs	Candidate	
POST	/api/applications/:id/withdraw	Candidate	
PATCH	/api/applications/:id/status	Owner	Moves the pipeline; appends to <code>statusHistory</code> ; emails the candidate.
POST	/api/companies	Recruiter	Create; unverified.
PATCH	/api/companies/:id	Owner	
GET	/api/companies/:slug	Public	Public company page + its active roles.
GET	/api/admin/users	Admin	Move the existing <code>/auth/users</code> here and add pagination.
PATCH	/api/admin/users/:id/role	Admin	
PATCH	/api/admin/users/:id/status	Admin	Activate / deactivate.
GET	/api/admin/companies?verified=false	Admin	Verification queue.

METHOD	PATH	ACCESS	PURPOSE
POST	/api/admin/companies/:id/verify	Admin	
GET	/api/admin/jobs? status=PENDING_REVIEW	Admin	Moderation queue.
POST	/api/admin/jobs/:id/approve	Admin	
POST	/api/admin/jobs/:id/reject	Admin	With a reason, emailed to the recruiter.

Build order

Six phases, each one shippable on its own. The sequencing rule: never build a screen before the endpoint that feeds it, and never build an endpoint that nothing on screen calls. Sizes assume solo work at a steady pace, not a sprint.

0 ~1 WEEK

Make the auth you have safe to expose

You are one deploy away from a public URL. Everything here is cheaper now than after someone finds the login form.

- Add `helmet`. Add `express-rate-limit`: login 5 per 15 min per IP+email, forgot-password 3/hour, register 10/hour per IP.
- Replace the per-controller `if (!x)` checks with `zod` schemas behind one `validate(schema)` middleware, so every 400 has the same shape.
- Wire a mail provider. Move the reset token out of `console.log` and into an email.
- Email verification end to end: token on register, `/verify-email/:token`, resend. Gate *applying* on it, not browsing.
- Hash refresh tokens before storing; restructure to the object form with device metadata.
- Account lockout after repeated failures.
- Either mount `getProfile` or delete it. Remove the `crypto` dependency. Add `CLIENT_URL` to `.env`.
- Structured logging (`pino`) with a request id on every line.
- Set up Vitest + Supertest and cover the auth flow. It stays cheap only if you start now.

DONE WHEN

A new user registers, receives a verification email, verifies, logs in, requests a reset, receives it, resets — and their old sessions stop working. Six bad passwords in a row locks the account.

1 ~1 WEEK

Candidate profile

The schema already describes a complete candidate. Give it a way in. Nothing downstream — applying, matching, recruiter review — means anything until this exists.

- `GET` and `PATCH /api/me/profile`, plus the experience and education sub-resources.
- Resume upload to object storage with a presigned URL. Cap at 5 MB, PDF/DOC only, and check the file's magic bytes — an extension is not a content type.

- Frontend `/profile` page, section by section, with the completeness meter reusing the `profileScore` helper already written for the admin table.

THE ONE THAT BITES EVERYONE

`PATCH /profile` must whitelist fields explicitly. If you pass `req.body` into `findByIdAndUpdate`, a candidate posts `{"role": "ADMIN"}` and owns your site. Pick fields off the body by name — never spread it.

DONE WHEN

A candidate can fill in every field the schema defines, upload a resume, reload, and see 100% completeness — and cannot change their own role, active flag or verified flag through any route.

2 ~2 WEEKS

Companies and jobs

The centre of the product, and the longest phase. Split it in two.

2a — write side

- `Company` and `Job` models with the status lifecycle.
- Recruiter registration path (role chosen at signup) and company creation.
- Recruiter dashboard: job list by status, create/edit form, publish, pause, close.

2b — read side

- `GET /api/jobs` with filters, sort and pagination. Mongo text index for keyword; compound indexes for the facets. `skip / limit` is fine well past your first ten thousand listings.
- Job detail page at `/jobs/:slug`.
- Swap `src/data/jobs.js` for an API layer. Keep the exported shapes so `JobBrowser`, `JobCard` and `FilterRail` need almost no change; move filtering server-side and let TanStack Query hold the results.
- Nightly job to expire listings past `expiresAt`.

DONE WHEN

A recruiter posts a role, an admin approves it, and it appears in candidate search with every filter narrowing correctly and pagination holding across filter changes.

3 ~1.5 WEEKS

Applications and saved jobs

This is the phase where the loop actually closes.

- `Application` model with snapshots, `statusHistory`, and the unique `(job, candidate)` index doing the duplicate-prevention rather than a race-prone check in your controller.
- Apply flow: cover letter, screening answers, resume confirmation. Increment `applicantCount` with `$inc` in the same operation.
- Candidate tracker at `/applications`, grouped by status.
- `SavedJob` collection; wire the bookmark button that currently forgets on refresh.
- Recruiter pipeline view per job, with status changes and a note.
- Email the candidate on every status change. This is the feature people stay for.

DONE WHEN

A candidate applies, cannot apply twice, sees it in their tracker; the recruiter shortlists them; the candidate gets an email and sees the new status with its timestamp.

4 ~3 DAYS

Admin write actions

The console is already laid out for this — it is read-only purely because the API is. Each endpoint lights up a control that already has a home in the table and the drawer.

- Role change, activate/deactivate, soft delete.
- Company verification queue and job moderation queue.
- Guard rails, enforced server-side: an admin cannot demote or deactivate themselves, and the last remaining admin cannot be removed.
- `AuditLog` on every write, surfaced in the console.

DONE WHEN

Every action in the admin UI works, is logged with an actor, and the self-lockout guards are proven by a test.

5 ~1 WEEK

Launch

Start this track in parallel at Phase 0 — deploy something empty early, so deployment is never the thing standing between you and users.

- API on a managed host, MongoDB on Atlas with backups on, frontend on a static host.

- Cross-site cookies need HTTPS on both origins and `sameSite: "none"; secure: true` — your `isProduction` switch already does this; what it needs is a correct `CLIENT_URL` and real certificates.
- Error tracking, uptime checks, and an alert that reaches you when `/health` goes red.
- Seed script with demo accounts for all three roles.
- Privacy policy, terms, and a working account-deletion route.

DONE WHEN

A stranger on a phone can register, verify, apply, and hear back — and you find out from a monitor, not from them, when something breaks.

Going live with real users

You chose real users over a portfolio demo, which changes what "finished" means. These are the obligations that come with that choice.

You are holding sensitive data

Resumes, phone numbers, salary expectations, employment history, notice periods — a job seeker's profile is among the most sensitive things an ordinary person will hand a website, and much of it can cost them their current job if it leaks. Practically, that means: resume files behind signed, expiring URLs rather than public buckets; a real account deletion path; never logging request bodies that carry profile data; and recruiters seeing only the candidates who applied to their own roles, never a browsable database of everyone. Data protection rules vary by country and I am not the right source on which apply to you — worth twenty minutes with someone who knows the rules where your users are.

Abuse is not hypothetical

- Fake job posts harvesting personal documents. Answer: company verification before a listing goes live.
- Scripted signups. Answer: email verification plus rate limits — both already in Phase 0.
- Credential stuffing on login. Answer: lockout and throttling.
- Malware uploaded as a "resume". Answer: type and size checks, magic-byte validation, serve from a separate origin, never execute or preview server-side.
- A reporting route on jobs and companies, even if triage is just you reading an inbox at first.

Operational minimums

- Secrets in the host's environment store, never in the repo. Rotate the JWT secrets before launch — anything that has sat in a dev `.env` should be considered spent.
- Atlas automated backups, and one restore rehearsed before you need it.
- Error tracking with source maps, so a frontend stack trace means something.
- A staging environment against a separate database. Testing against production data goes wrong exactly once.

Decisions to make before Phase 1

Each of these blocks a phase. Pick fast — none is hard to change later except the first.

DECISION	OPTIONS	LEANING
File storage	Cloudinary · AWS S3 · UploadThing	S3 with presigned URLs if you want the infra experience and precise access control over resumes; Cloudinary if you would rather spend the week on product.
Email	Resend · SendGrid · AWS SES	Start with whichever you can get sending in an hour. Use your own domain from day one — deliverability on a shared sender is poor, and password resets landing in spam looks like an outage.
Hosting	Render / Railway / Fly for the API · Vercel or Netlify for the frontend · Atlas for data	Any of them. Confirm cross-site cookies work on your chosen pair before you build on it — that is the one thing that differs between them in a way that will cost you a day.
Search	Mongo text index · Atlas Search · Meilisearch	Text index now. It is one line and good enough for thousands of listings. Move only when relevance, not speed, becomes the complaint.
Validation	zod · Joi · express-validator	zod — the same schema can type your frontend forms.
Frontend data	TanStack Query everywhere · keep the hand-rolled auth fetch	Query is already installed and doing the admin directory. Move the auth calls onto it too rather than maintaining two patterns.

What will actually bite you

The cold-start problem, not the code

A job board with no jobs gets no candidates; with no candidates it gets no recruiters. This is the risk that kills job boards, and no amount of engineering answers it. The usual way through is to pick something narrow — one city, one discipline, one community you are already in — seed the first twenty roles by hand from companies you can call, and stay narrow far longer than feels comfortable. Decide this before Phase 2, because it changes what you build: a board for one city needs no country filter.

Scope, in the specific place it will slip

The recruiter side roughly doubles the surface area of the MVP. If Phase 2 runs long, the cut that costs you least is the posting UI: keep the `Job` and `Company` models exactly as specced, create listings yourself through the admin console, and ship the candidate experience. The data model does not change, so the recruiter portal drops in later with nothing to migrate.

Denormalised counters drift

`applicantCount` will disagree with reality the first time an application is created and the increment fails. Update both in one operation, and run a nightly reconciliation that recomputes counters from the source collection.

Snapshots are not optional

If `Application` merely references the live profile, a candidate who tidies their resume in March silently rewrites what a recruiter saw in January. Copy the fields at apply time.

Test the guards, not the getters

The tests worth writing are the ones asserting what must *not* happen: a candidate cannot read another candidate's application, a recruiter cannot see applicants for someone else's job, a user cannot promote themselves, an admin cannot remove the last admin. Those four tests are worth more than fifty covering happy paths.

Written against the state of `node-auth` and `node-auth-frontend` on 7 September 2026. Estimates assume solo work at a steady pace. Phases are ordered by dependency — where two are independent, build the one that puts something in front of a user first.